

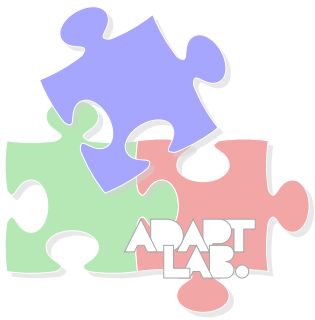
Employing Metaprogramming: A Macro-Driven Strategy to Overcome Compiler-Imposed Limitations on Specialization

Davide Cerato
Id. Number: 984481

Università degli Studi di Milano
Computer Science Department
BSc in Computer Science

Advisor: Prof. Walter Cazzola
Co-Advisor: M.Sc. Federico Bruzzone

L-31 - Computer Science
Academic Year 2024-2025





Specialization
State of the Art and Limitations

Specialization

Problem Statement

Proposal

By Example: Implementing the Proposal

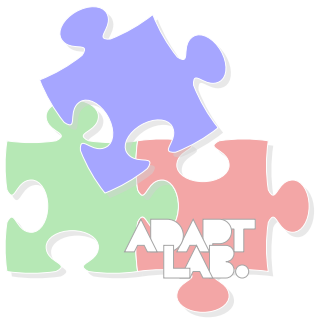
Implementation Details

Limitations

Evaluation

Conclusions

Systems programming aims to combine generic programming with type-specific optimizations.





Specialization
State of the Art and Limitations

Specialization

Problem Statement

Proposal

By Example: Implementing the Proposal

Implementation Details

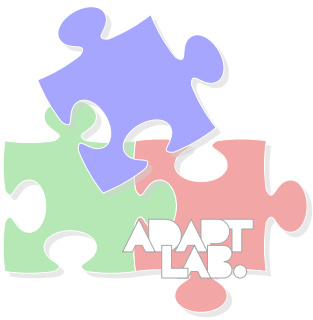
Limitations

Evaluation

Conclusions

Systems programming aims to combine generic programming with type-specific optimizations.

Mechanisms vary widely, from mature and widely used systems to experimental features, with varying degrees of success.





Specialization
State of the Art and Limitations

Specialization

Problem Statement

Proposal

By Example: Implementing the Proposal

Implementation Details

Limitations

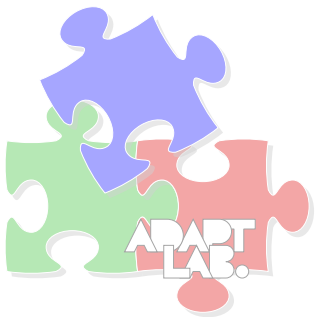
Evaluation

Conclusions

Systems programming aims to combine generic programming with type-specific optimizations.

Mechanisms vary widely, from mature and widely used systems to experimental features, with varying degrees of success.

In many modern ecosystems, specialization is often unstable, forcing a tradeoff between performance and compiler stability.





Problem Statement
What's happening in Rust?

In 2015, an RFC introduced specialization to Rust as an unstable feature confined to the Nightly Channel.





Problem Statement
What's happening in Rust?

In 2015, an RFC introduced specialization to Rust as an unstable feature confined to the Nightly Channel.

However, due to unresolved design issues and concerns about soundness of the type system, specialization is still unstable.



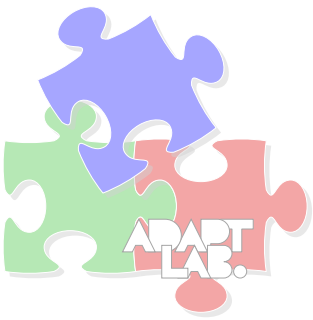


Problem Statement
What's happening in Rust?

In 2015, an RFC introduced specialization to Rust as an unstable feature confined to the Nightly Channel.

However, due to unresolved design issues and concerns about soundness of the type system, specialization is still unstable.

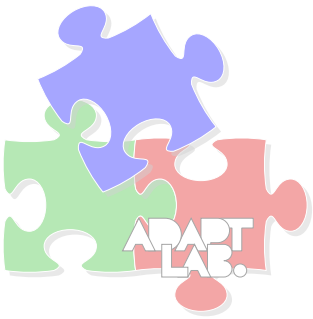
Without stable specialization, developers often resort to code duplication or complex workarounds.





Problem Statement
Motivational Example

Consider implementing a trait for different types, with a specialized implementation for a specific type.

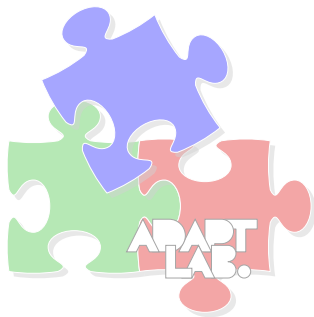




Problem Statement
Motivational Example

Consider implementing a trait for different types, with a specialized implementation for a specific type.

```
1  trait MyTrait { fn my_method(&self); }
2  impl<T> MyTrait for T {
3      fn my_method(&self) {
4          println!("Generic implementation");
5      }
6  }
7  impl MyTrait for i32 {
8      fn my_method(&self) {
9          println!("Specialized for i32");
10     }
11 }
12 fn main() {
13     10.my_method();                // should print "Specialized for i32"
14     10.5.my_method();              // should print "Generic implementation"
15 }
```



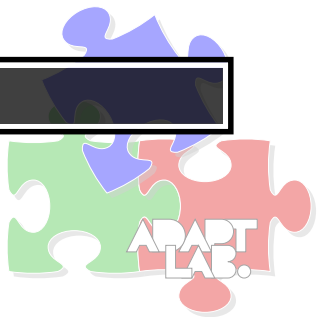


Problem Statement
Motivational Example

Consider implementing a trait for different types, with a specialized implementation for a specific type.

```
1  trait MyTrait { fn my_method(&self); }
2  impl<T> MyTrait for T {
3      fn my_method(&self) {
4          println!("Generic implementation");
5      }
6  }
7  impl MyTrait for i32 {
8      fn my_method(&self) {
9          println!("Specialized for i32");
10     }
11 }
12 fn main() {
13     10.my_method();                // should print "Specialized for i32"
14     10.5.my_method();              // should print "Generic implementation"
15 }
```

```
error[E0119]: conflicting implementations of trait MyTrait for type i32
```

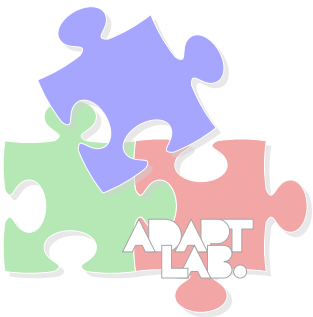




Proposal
Macro-Based Specialization

Employ the powerful metaprogramming capabilities of Rust to build a macro library that simulates native specialization, with:

- Native-like look and feel
- Compile-time resolution
- Sound and stable
- Zero runtime overhead
- Powerful API



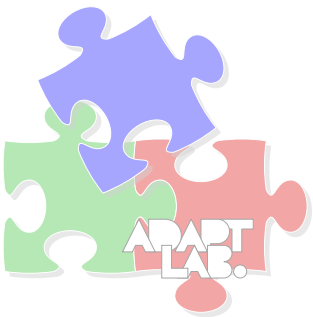


Proposal
Macro-Based Specialization

Employ the powerful metaprogramming capabilities of Rust to build a macro library that simulates native specialization, with:

- Native-like look and feel
- Compile-time resolution
- Sound and stable
- Zero runtime overhead
- Powerful API

To do this, we use two macros: `#when` and `spec!`.





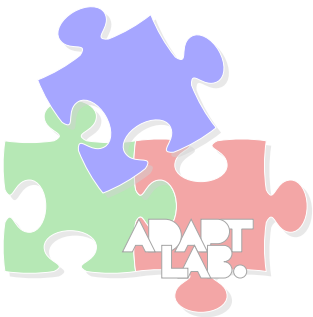
Proposal
Macro-Based Specialization

Employ the powerful metaprogramming capabilities of Rust to build a macro library that simulates native specialization, with:

- Native-like look and feel
- Compile-time resolution
- Sound and stable
- Zero runtime overhead
- Powerful API

To do this, we use two macros: `#when` and `spec!`.

The approach can also be applied to other languages or even directly into the compiler frontend.



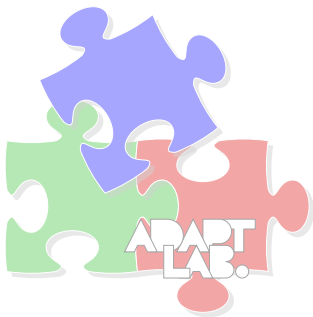


By Example: Implementing the Proposal

Motivational Example

Let's revisit the motivational example we presented earlier.

```
1  trait MyTrait { fn my_method(&self); }
2  impl<T> MyTrait for T {
3      fn my_method(&self) {
4          println!("Generic implementation");
5      }
6  }
7  impl MyTrait for i32 {
8      fn my_method(&self) {
9          println!("Specialized for i32");
10     }
11 }
12 fn main() {
13     10.my_method();                // should print "Specialized for i32"
14     10.5.my_method();              // should print "Generic implementation"
15 }
```

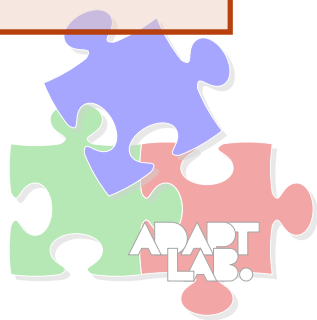




By Example: Implementing the Proposal
Motivational Example

Let's revisit the motivational example we presented earlier.

```
1  trait MyTrait { fn my_method(&self); }
2  impl<T> MyTrait for T {
3      fn my_method(&self) {
4          println!("Generic implementation");
5      }
6  }
7  #[when(T = i32)]
8  impl<T> MyTrait for T {
9      fn my_method(&self) {
10         println!("Specialized for i32");
11     }
12 }
13 fn main() {
14     spec! { 10.my_method(); i32 };        // prints "Specialized for i32"
15     spec! { 10.5.my_method(); f64 };      // prints "Generic implementation"
16 }
```



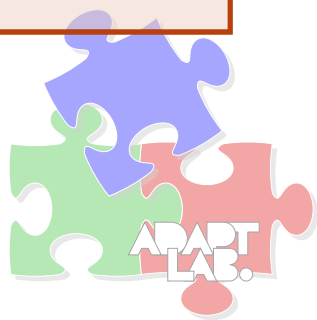


By Example: Implementing the Proposal

Motivational Example

Let's revisit the motivational example we presented earlier.

```
1  trait MyTrait { fn my_method(&self); }
2  impl<T> MyTrait for T {
3      fn my_method(&self) {
4          println!("Generic implementation");
5      }
6  }
7  trait MyTrait_i32 { fn my_method(&self); }
8  impl MyTrait_i32 for i32 {
9      fn my_method(&self) {
10         println!("Specialized for i32");
11     }
12 }
13 fn main() {
14     <i32 as MyTrait_i32>::my_method(&10); // prints "Specialized for i32"
15     <f64 as MyTrait>::my_method(&10.5);   // prints "Generic implementation"
16 }
```

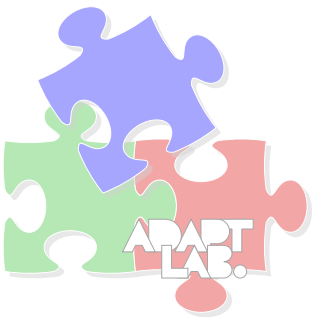




By Example: Implementing the Proposal
Supported Predicates and Forms

The following predicates are supported:

- `T = Type`
- `all(p1, p2, ...)`
- `not(p)`
- `T: Tr1 + Tr2`
- `any(p1, p2, ...)`





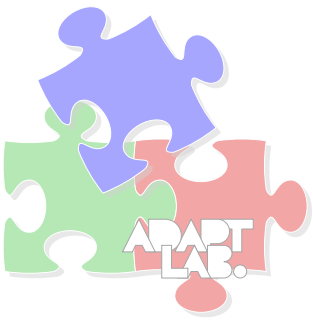
By Example: Implementing the Proposal
Supported Predicates and Forms

The following predicates are supported:

- `T = Type`
- `all(p1, p2, ...)`
- `not(p)`
- `T: Tr1 + Tr2`
- `any(p1, p2, ...)`

We also support the following implementations forms:

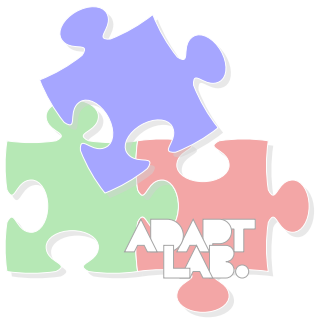
- `impl<T> Trait for T`
- `impl<T> Trait for Type<T>`
- `impl<T> Trait<T> for Type`





By Example: Implementing the Proposal
Lifetimes

Lifetimes are one of the main reasons native specialization is still unstable.

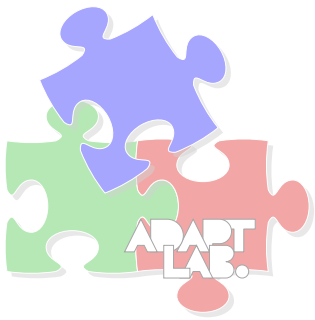




By Example: Implementing the Proposal
Lifetimes

Lifetimes are one of the main reasons native specialization is still unstable.

Lifetimes bounds can be implicit or explicit.



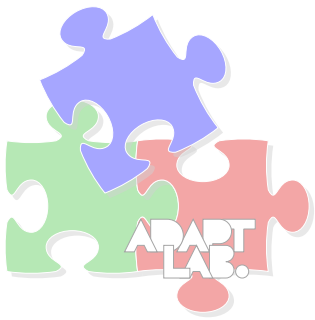


By Example: Implementing the Proposal
Lifetimes

Lifetimes are one of the main reasons native specialization is still unstable.

Lifetimes bounds can be implicit or explicit.

In our belief, this is an architectural limitation of the compiler that can be overcome with proper re-design.





By Example: Implementing the Proposal
Lifetimes

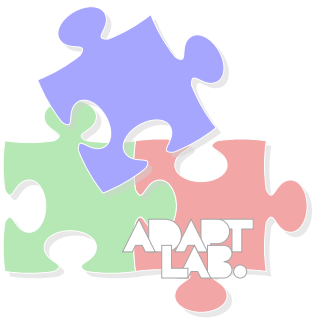
Lifetimes are one of the main reasons native specialization is still unstable.

Lifetimes bounds can be implicit or explicit.

In our belief, this is an architectural limitation of the compiler that can be overcome with proper re-design.

Our proposal treats lifetimes as first-class citizens:

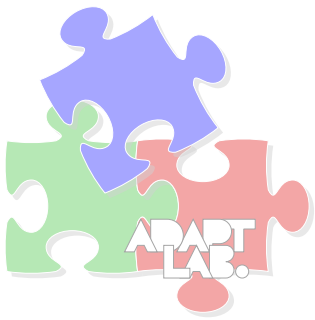
- `T = &'a _`
- `T: 'a`





By Example: Implementing the Proposal
Higher-Ranked Polymorphism

Our macro system also supports Higher-Ranked Polymorphism.

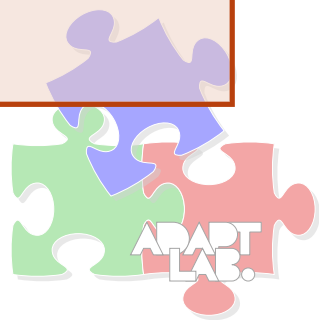




By Example: Implementing the Proposal
Higher-Ranked Polymorphism

Our macro system also supports Higher-Ranked Polymorphism.

```
1  struct S;
2  trait Trait<T, U, V> {
3      fn f(&self, t: T, u: U) -> V;
4  }
5  #[when(all(
6      T = &str, T: 'b,
7      U = for<'a> fn(T, &'a i32) -> V
8  ))]
9  impl<'b, T, U, V> Trait<T, U, V> for S {
10     fn f(&self, t: T, u: U) -> V {
11         u(t, &7)
12     }
13 }
14 fn main() {
15     let s = S;
16     let lambda = |a: &str, b: &i32| a.len() as i32 + *b;
17     let res: i32 = spec! { s.f("abc", lambda); S;
18       [&str, for<'a> fn(&str, &'a i32) -> i32] }; // returns 10
19 }
```

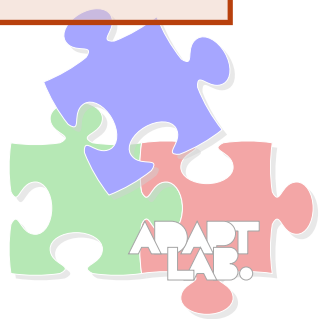




By Example: Implementing the Proposal
Higher-Ranked Polymorphism

Our macro system also supports Higher-Ranked Polymorphism.

```
1  struct S;
2  trait Trait<T, U, V> {
3      fn f(&self, t: T, u: U) -> V;
4  }
5  trait Trait_fn<'b, V> {
6      fn f(&self, t: &'b str, u: for<'a> fn(&'b str, &'a i32) -> V) -> V;
7  }
8  impl<'b, V> Trait_fn<'b, V> for S {
9      fn f(&self, t: &'b str, u: for<'a> fn(&'b str, &'a i32) -> V) -> V {
10         u(t, &7)
11     }
12 }
13 fn main() {
14     let s = S;
15     let lambda = |a: &str, b: &i32| a.len() as i32 + *b;
16     let res: i32 = <S as Trait_fn<_>>::f(&s, "abc", lambda); // returns 10
17 }
```

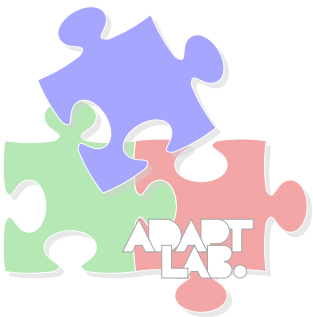




Implementation Details
Handling Conditional Implementations

Conditions defined in the `when` macro are normalized into DNF, flattening when possible, e.g.:

```
all(T = Vec<U>, any(U = i32, U = f64)) → any(T = Vec<i32>, T = Vec<f64>)
```





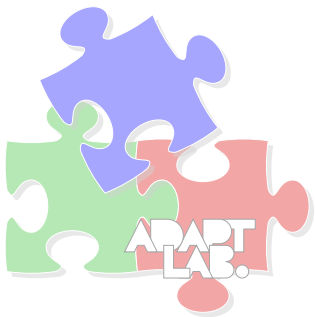
Implementation Details
Handling Conditional Implementations

Conditions defined in the `when` macro are normalized into DNF, flattening when possible, e.g.:

```
all(T = Vec<U>, any(U = i32, U = f64)) → any(T = Vec<i32>, T = Vec<f64>)
```

A partial order defined over the predicates is used to determine the specificity of each implementation, e.g.:

```
T = &i32 > T = &U > T: Copy
```





Implementation Details
Handling Conditional Implementations

Conditions defined in the `when` macro are normalized into DNF, flattening when possible, e.g.:

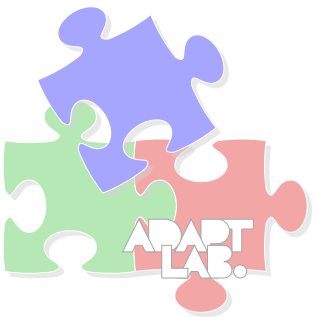
```
all(T = Vec<U>, any(U = i32, U = f64)) → any(T = Vec<i32>, T = Vec<f64>)
```

A partial order defined over the predicates is used to determine the specificity of each implementation, e.g.:

```
T = &i32 > T = &U > T: Copy
```

We need a way to collect all the traits defined in the crate and to handle the order of their implementations. In Rust, this is done with Build Scripts, e.g.:

```
mono_spec::handle_order();
```





Limitations
Some Shortcomings

The usage of macros makes the tool flexible and powerful, but it also introduces some limitations and shortcomings, e.g.:

- Boilerplate
- Compile-Time Overhead
- Language Features Support
- Same Project Traits
- Warnings and Errors
- Trait Bounds





Evaluation
Approach

Once identified some patterns, we instrumented the compiler to analyze at HIR level any function definition.

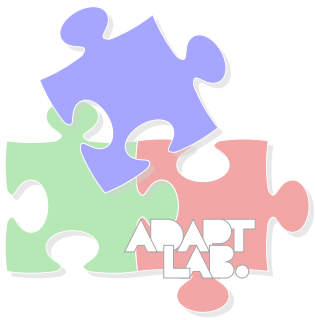




Evaluation
Approach

Once identified some patterns, we instrumented the compiler to analyze at HIR level any function definition.

For each one, we created a custom AST and using heuristics, we grouped functions that could be mutually specializable.





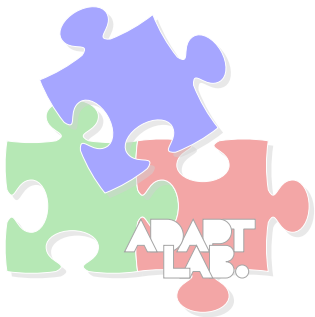
Evaluation
Approach

Once identified some patterns, we instrumented the compiler to analyze at HIR level any function definition.

For each one, we created a custom AST and using heuristics, we grouped functions that could be mutually specializable.

We then calculated the tree edit distance using the ZSS algorithm and from that, we derived a similarity score:

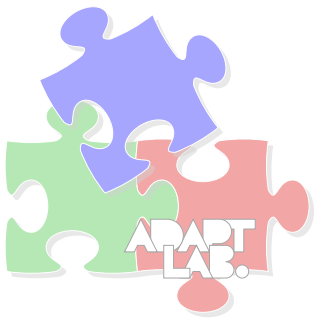
$$1 - \frac{\text{dist}}{\text{max size}}$$





Evaluation
Results

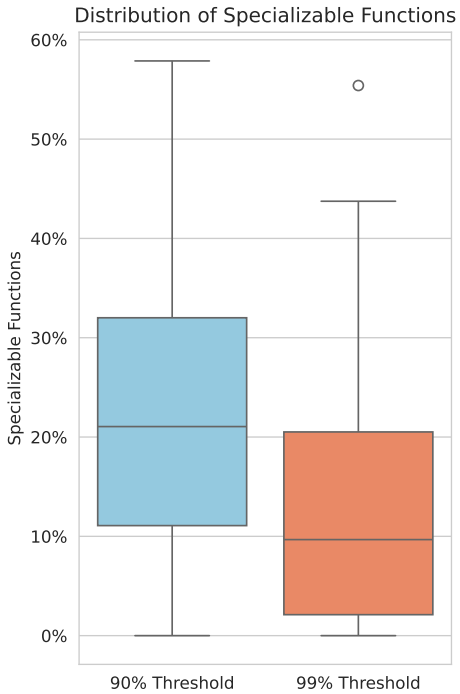
The analysis was run twice on 65 open-source Rust codebases: with a similarity threshold of 90% and 99% respectively.

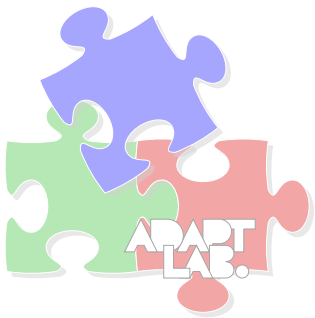




Evaluation
Results

The analysis was run twice on 65 open-source Rust codebases: with a similarity threshold of 90% and 99% respectively.

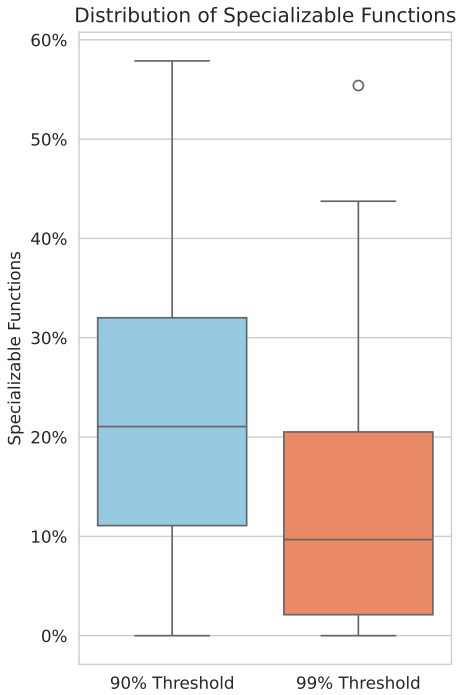


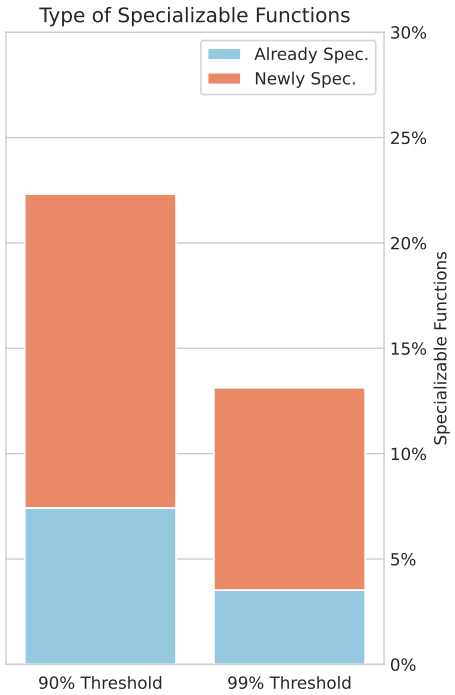


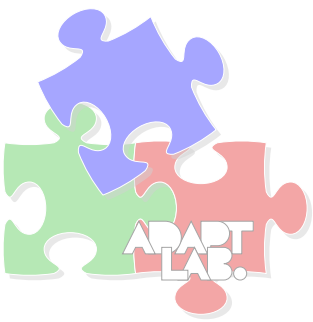


Evaluation
Results

The analysis was run twice on 65 open-source Rust codebases: with a similarity threshold of 90% and 99% respectively.



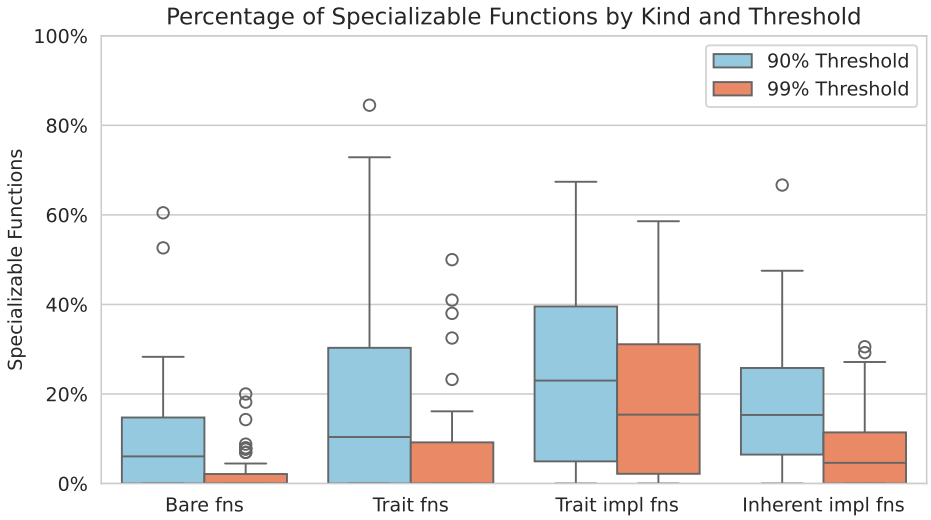


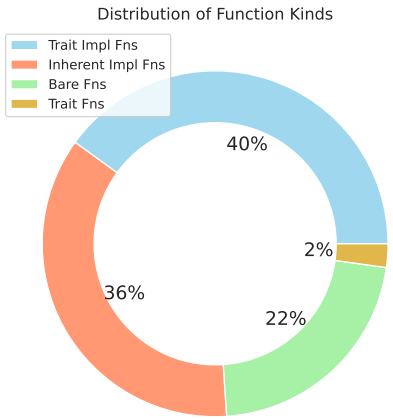


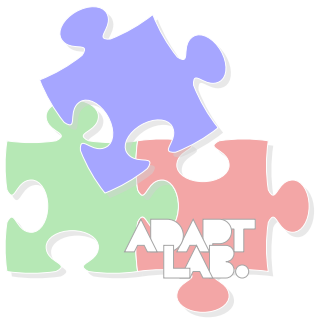


Evaluation
Results

We then analyzed the kinds of functions that were found to be specializable.









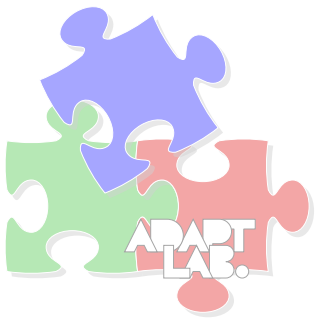
Conclusions

Interesting results:

- We are writing an article (Meta-Monomorphizing Specializations), to be submitted to ECOOP.

Future work:

- Extend the tool to support other language features and function kinds.





Thanks for your attention!

